Project to replace modal-requirements with a new and more intuitive modal. Right now I don't want to touch my previous code, so we just gonna comment where we call for the old modal and there we gonna add the new modal. All the code .js and .css for this new modal gonna sit in other files.

1. Technical Details & Requirements

This section defines the core architecture and logic rules for the new modal. We will treat this like a lightweight Single Page Application (SPA) inside the modal itself.

A. State Management (The Brain)

We need to manage two distinct states:

- **The Navigation Stack (currentPath)**: An array tracking the user's drill-down journey. Example: ['Company Name', 'ISO 9001:2015', '8', '8.3'].
- **The Selection Bucket (markedClauses)**: An object storing the actual requirements the user has formally "Marked" for this specific process, grouped by standard. Example: {"ISO 9001:2015": ["4.1", "8"], "ISO 14001:2015": ["4.1"]}.

B. The Navigation Engine (Left Column)

- **Active State:** The last item in the Navigation Stack is the "Active" SP Box.
- **Options Area:** The grid below the stack will dynamically render the *immediate children* of the Active SP Box. If the user clicks standard ISO 9001, the Options Area shows Chapters 4 through 10.
- **The "X" (Pop) Logic:** Clicking the "X" on an SP Box in the stack acts as a "Back" button. If the stack is [Company, ISO 9001, 8, 8.1] and the user clicks the "X" on 8, the stack pops back to [Company, ISO 9001]. Both 8 and 8.1 are removed from the stack, and the Options Area resets to show the chapters.
- **Selecting vs. Marking:** * *Selecting* (clicking the SP Box body) = Navigation. It pushes the item to the stack and updates the Right Column.
    - *Marking* (clicking a specific "Add/Check" icon on the SP Box) = Action. It adds the clause to the Selection Bucket (Row 3).

C. The Content Viewer (Right Column)

- **Dynamic Binding:** This column simply listens to the Active State. Whenever the last item in the Navigation Stack changes, this column fetches and renders the corresponding HTML content from your normative database.
- **Formatting:** We will use a scoped CSS class (e.g., .new-clause-text-flow) to ensure the text, bolding, and sub-clauses render beautifully without inheriting conflicts from the old CSS. Scrollbars will be styled to be subtle and modern.

D. The Selection Bucket & Grandparent Logic (Row 3)

- **Data Grouping:** Elements in Row 3 will be visually grouped by their Standard using mini-headers or tags.
- **The Grandparent Filter:** Before rendering Row 3, the state must pass through a reduction algorithm. We will extract the logic you already have in script.js (likely a descendant-checking function). If the user marks 4, 4.1, and 4.1.2, the UI will *only* display 4.
- **Reverse Navigation:** Clicking an SP Box inside Row 3 will instantly reconstruct the Navigation Stack. If the user clicks 8.3 in Row 3, the stack automatically builds [Company, Standard, 8, 8.3] and updates the Left and Right columns accordingly.
- **Unmarking:** Clicking the "X" on an SP Box in Row 3 removes it from the markedClauses state and triggers a re-render of the bucket.

---

2. Construction Plan (Phases)

To ensure we don't end up with spaghetti code, we will build this systematically. Each phase will result in a fully functional, testable block before moving to the next.

Phase 1: The UI Skeleton & Layout (HTML/CSS)

- **Goal:** Create the visual shell of the modal using CSS Grid and Flexbox. No Javascript yet.
- **Tasks:** * Build Row 1 (Header + Editable Title + Close Modal).
    - Build Row 2 structure (Left Col / Right Col layout).
    - Build Row 3 structure (Bottom bucket).
    - Design the CSS for the "SP Box" (compact, rounded corners, hover states, icons).

Phase 2: The State & Mock Navigation (JS)

- **Goal:** Implement the Navigation Stack logic using mock data.
- **Tasks:**
    - Create the currentPath array state.
    - Write the render function for the Left Column stack (displaying the fixed Company Name and looping through the path array).
    - Write the render function for the Options Area (hardcode a few levels of children just to test clicking and drilling down).
    - Implement the "X" (pop stack) logic.

Phase 3: Database Hookup & Right Column Content (JS)

- **Goal:** Connect the Left Column to your actual NORM_DATA and render the reading text.
- **Tasks:**
    - Replace the mock children in the Options Area with real queries to your database.
    - Write the render function for the Right Column.
    - Ensure that when an SP Box is clicked on the left, the exact, formatted text appears on the right.

Phase 4: The Selection Bucket & Grandparent Filter (JS)

- **Goal:** Implement the "Marking" action and the Row 3 display.
- **Tasks:**
    - Add the "Mark" icon to the SP Boxes in the Options Area.
    - Create the markedClauses state.
    - Implement the "Grandparent" filtering logic so redundant children are hidden.
    - Render Row 3, grouping the marked items by standard.
    - Implement the "Unmark" (remove) action.

Phase 5: Reverse Navigation & Polish (JS/CSS)

- **Goal:** Make the modal fully interactive and smooth.
- **Tasks:**
    - Make the SP Boxes in Row 3 clickable, so clicking one auto-generates the Navigation Stack in the Left Column.
    - Make the Process Name in Row 1 editable.
    - Final CSS polish (transitions, scrollbar styling, ensuring it looks good on different screen sizes).